PWM Breathing LED

1. PWM Overview

1.1 What is PWM

PWM (Pulse Width Modulation) is simply a technique that "uses digital switches to simulate analog voltage".

In digital circuits, GPIO pins can only output two states: high (3.3V) or low (0V), no intermediate values. But PWM cleverly bypasses this limitation through "rapid switching"—the faster the switching, and with different ratios of on-time, the resulting "average effect" on the load differs.

Think of it this way: you can't make a water tap produce "half-thick" water column, but you can rapidly turn it on and off, 50% on / 50% off, and the bucket receives only half the water of a fully open tap. PWM is this concept.

1.2 Three Core Parameters of PWM

① **Period**: Time for one complete "on→off" cycle, in seconds. Period = 1 / frequency.

② **Frequency**: How many "on→off" cycles per second, in Hz. How high should frequency be?

- **Too low (<50Hz)**: Human eye can detect LED flickering, motor vibrates
- **Moderate (~1kHz)**: LED appears stable, ordinary motors run smoothly—this experiment is 1kHz

- **Too high (>20kHz)**: Beyond human hearing range, suitable for quiet motor drive scenarios, but switching losses increase

③ **Duty Cycle**: In one cycle, the ratio of "high level time" to "entire cycle time".

```
Duty cycle = (high level time / period) × 100%

Example frequency 1kHz (period = 1ms):
  25% duty cycle  = high 0.25ms + low 0.75ms  → average voltage ≈ 0.825V
  50% duty cycle  = high 0.50ms + low 0.50ms  → average voltage = 1.65V
  80% duty cycle  = high 0.80ms + low 0.20ms  → average voltage = 2.64V
```

PWM's magic is: **You only change the "on/off" time ratio (duty cycle), but the load feels like voltage is continuously changing**. This is the "area equivalence principle"—the average value of high-frequency square wave equals a constant analog voltage value.

1.3 What This Experiment Does

This experiment uses ESP32-S3's **LEDC (LED PWM Controller)** peripheral to generate PWM signals, driving an LED to display **"breathing light" effect** by dynamically changing duty cycle: gradually bright → gradually dim → off stay → repeat.

Application Scenarios:

Scenario	Description
LED brightness control	Breathing light, keyboard backlight, ambiance light
Motor speed control	Servo angle, DC motor speed
Audio output	Generate analog audio with low-pass filter
Power management	DC-DC converter control signal

---

2. Core Concepts

2.1 Duty Cycle and Resolution

```
PWM signal (1kHz frequency, period = 1ms):

  0% duty cycle:   ──────────────────────────────  average voltage = 0V
                  (entire low level)

  25% duty cycle:  ┌──┐┌──────────────────────────  average voltage = 0.825V
                   └──┘└──────────────────────────

  50% duty cycle:  ┌────┐┌──────────────────────────  average voltage = 1.65V
                   └────┘└──────────────────────────

  100% duty cycle: ┌──────────────────────────────  average voltage = 3.3V
```

```
                                    └──────────────────────────────┘
```

Duty cycle resolution: `LEDC_TIMER_10_BIT` means duty cycle divided into 0~1023 (1024 levels total). With 10-bit resolution:

Duty Value	Meaning (common-anode this experiment)
1023 (MAX_DUTY)	High level 100% = LED completely OFF
511	High level ~50% = LED half bright
200 (VISIBLE_DUTY)	High level ~20% = LED relatively bright (low level 80%)
0	High level 0% = LED brightest (all low level)

2.2 Common-Anode LED PWM Control

```
Common-anode connection:
  VDD (3.3V) ──→ LED anode ──→ LED cathode ──→ Current-limiting resistor ──→ GPIO44

  GPIO output HIGH (1023): LED both ends at same potential, no current → OFF
  GPIO output LOW  (0):   Current VDD→LED→GPIO→GND  → ON
```

This experiment's "reverse brightness" relationship:

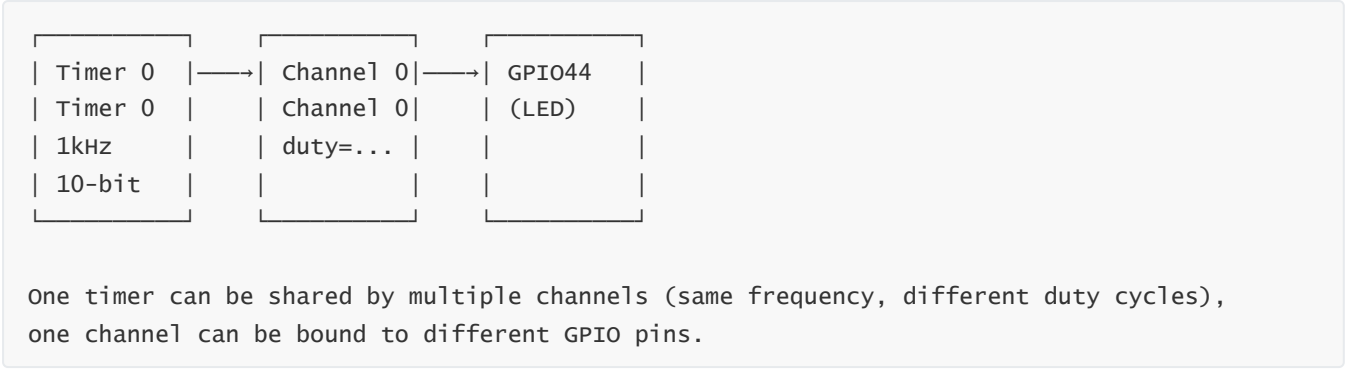
duty value	Low level ratio	LED brightness	State description
1023	0%	**OFF**	OFF phase stay
1023→200	0%→80%	**Gradually bright**	fade-in
200→1023	80%→0%	**Gradually dim**	fade-out

> In common-anode connection, duty decreases → low level ratio increases → LED gets brighter. This is opposite to the conventional understanding "duty increases = brighter", so code uses `MAX_DUTY` (1023) for OFF, `VISIBLE_DUTY` (200) for visible brightness.

2.3 Why VISIBLE_DUTY = 200 Instead of 0

If duty drops to 0 (100% low level = brightest), LED will instantly jump from "fully bright" to "fully off" (phase switch), visually discontinuous. 200 provides a "brightest but not fully bright" visible threshold, forming brightness contrast with MAX_DUTY, making breathing effect smooth.

2.4 LEDC Peripheral Architecture



One timer can be shared by multiple channels (same frequency, different duty cycles), one channel can be bound to different GPIO pins.

LEDC Resources:

Resource Type	Available Count	Description
Timer	4	Determines PWM frequency and resolution
Channel	8	Bind GPIO and duty cycle, share timer
Speed mode	Low Speed / High Speed	Low speed suitable for LED, high speed for precise edge requirements

3. Project Architecture and Flow

```
app_main()
  ├─ led_pwm_init()              Initialize LEDC timer + channel
  |    ├─ ledc_timer_config()       Timer: 1kHz, 10-bit, low speed mode
  |    ├─ ledc_channel_config()     Channel 0: bind GPIO44, initial duty=0
  |    └─ ledc_fade_func_install()  Install fade function (not used in this experiment)
  |
  ├─ xTaskCreate()               Create breathing light independent task (stack 2048B,
priority 1)
  |
  └─ while(1)                    Main loop suspended (idle)
       └─ vTaskDelay(1000ms)

Breathing light task (independent FreeRTOS task):
  breath_led_task()
    while(1):
      ├─ OFF Phase: duty = MAX_DUTY (1023), stay 2000ms
      ├─ OFF→ON:   duty from 1023 decrements to 200 (gradually bright)
      ├─ ON→OFF:   duty from 200 increments to 1023 (gradually dim)
      └─ Each step delays 12ms
```

> **Why put breathing light in an independent task?** Breathing light needs precise duty cycle updates and delay loops. Putting it in main task would block other operations. Independent task has priority 1 (lowest), doesn't affect more important system tasks.

4. Hardware Dependencies

Hardware Connection:

Pin	Peripheral	Description
GPIO44	LED	Common-anode, low level turns on. LEDC_CHANNEL_0 outputs PWM

> ⚠ **Pin conflict warning**: GPIO44 is also **UART0 TX** pin. If this experiment simultaneously uses `printf()` or `ESP_LOGI()`, PWM signal will interfere with serial TX output causing garbled text. Therefore **all printing is disabled** in this experiment—no `printf` or `ESP_LOGI` calls in `app_main`.

Dependencies: `freertos`, `driver/ledc`, `driver/gpio`

---

5. Source Code Analysis

5.1 Macro Definitions

Macro definitions essentially give "nicknames" to hardware parameters and values. First, convenient to change parameters in one place later. Second, writing `LED0_GPIO_PIN` in code is much more understandable than `GPIO_NUM_44`—at a glance you know it's the pin controlling LED.

```
#define LED0_GPIO_PIN    GPIO_NUM_44      // LED pin
#define LEDC_TIMER       LEDC_TIMER_0     // Use Timer 0
// Low speed mode (suitable for LED)
#define LEDC_MODE        LEDC_LOW_SPEED_MODE
#define LEDC_CHANNEL0    LEDC_CHANNEL_0   // Use Channel 0
// 10-bit resolution (0~1023)
#define LEDC_DUTY_RES    LEDC_TIMER_10_BIT
#define LEDC_FREQ_HZ     1000

// Duty step: increment/decrement by 5 each step
#define BREATH_STEP      5
#define BREATH_DELAY_MS  12               // Delay 12ms per step
// OFF state stay 2000ms
#define OFF_HOLD_MS      2000

// 1023 = OFF (all high level)
#define MAX_DUTY         ((1 << LEDC_DUTY_RES) - 1)
// Visible brightness threshold (<this value = ON)
#define VISIBLE_DUTY     200
```

Breathing speed calculation:

```
Each fade phase steps = (1023 - 200) / 5 = 165 steps
Each step delay = 12ms
fade-in time = 165 × 12ms ≈ 1980ms
fade-out time = 1980ms
OFF stay = 2000ms
Complete cycle ≈ 1980 + 1980 + 2000 ≈ 5960ms ≈ 6 seconds
```

5.2 led_pwm_init() — LEDC Initialization

Initialization function does two things: first configure "rhythm", then configure "route". Timer determines PWM speed and precision (1kHz, 10-bit), channel routes timer-generated signal to specific GPIO pin. Like first setting drum beat rhythm, then sending drum sound to designated speaker.

```c
void led_pwm_init(void)
{
    // === Configure Timer: decide PWM frequency and resolution ===
    ledc_timer_config_t ledc_timer = {
        // 10-bit resolution
        .duty_resolution = LEDC_DUTY_RES,
        .freq_hz         = LEDC_FREQ_HZ,      // 1kHz frequency
        .speed_mode      = LEDC_MODE,         // Low speed mode
        .timer_num       = LEDC_TIMER,        // Timer 0
        // Auto-select clock source
        .clk_cfg         = LEDC_AUTO_CLK,
    };
    ledc_timer_config(&ledc_timer);

    // === Configure Channel: bind timer's PWM signal to GPIO pin ===
    ledc_channel_config_t ledc_channel0 = {
        .channel    = LEDC_CHANNEL0,
        // Initial duty 0 (LED fully on)
        .duty       = 0,
        .gpio_num   = LED0_GPIO_PIN,
        .speed_mode = LEDC_MODE,
        .hpoint     = 0,                     // No phase offset
        .timer_sel  = LEDC_TIMER,
    };
    ledc_channel_config(&ledc_channel0);

    // Install hardware fade function (backup)
    ledc_fade_func_install(0);
}
```

> `ledc_fade_func_install(0)` installs hardware fade function, but this experiment doesn't use it—breathing effect is implemented by FreeRTOS task **software step-adjusting duty cycle**. Installing this function reserves for future example expansion.

> `LEDC_AUTO_CLK` lets driver automatically select appropriate clock source, no manual frequency division calculation needed.

5.3 breath_led_task() — Breathing Light Algorithm (Core)

Breathing light essence is an infinite loop running three "actions": first let LED stay off for a while → slowly push brightness to brightest → slowly drop brightness to darkest, repeat. After each duty change, wait a bit before next change—this is like animation principle, frame by frame, looks smooth to human eye.

```
void v_breath_led_task(void *pvParameters)
{
    while (1) {
        // === OFF Phase: LED fully off, hold OFF_HOLD_MS milliseconds ===
        // Set duty to max (LED off)
        ledc_set_duty(LEDC_MODE, LEDC_CHANNEL0, MAX_DUTY);
        // Push to hardware
        ledc_update_duty(LEDC_MODE, LEDC_CHANNEL0);
        // 2000ms hold
        vTaskDelay(pdMS_TO_TICKS(OFF_HOLD_MS));

        // === OFF→ON (fade in): duty decreases from large to small ===
        for (int32_t d = MAX_DUTY; d >= VISIBLE_DUTY; d -= BREATH_STEP) {
            // Set target duty
            ledc_set_duty(LEDC_MODE, LEDC_CHANNEL0, d);
            // Push to hardware
            ledc_update_duty(LEDC_MODE, LEDC_CHANNEL0);
            // Wait 12ms before next step
            vTaskDelay(pdMS_TO_TICKS(BREATH_DELAY_MS));
        }

        // === ON→OFF (fade out): duty increases from small to large ===
        for (int32_t d = VISIBLE_DUTY; d <= MAX_DUTY; d += BREATH_STEP) {
            // Set target duty
            ledc_set_duty(LEDC_MODE, LEDC_CHANNEL0, d);
            // Push to hardware
            ledc_update_duty(LEDC_MODE, LEDC_CHANNEL0);
            // Wait 12ms before next step
            vTaskDelay(pdMS_TO_TICKS(BREATH_DELAY_MS));
        }
    }
}
```

> **Why `ledc_set_duty()` + `ledc_update_duty()` in two steps?** This is ESP32-S3 LEDC's design convention. `set_duty` only modifies target duty in software register, `update_duty` triggers hardware to load new value at next PWM cycle start. This split design allows pre-setting multiple channels' duty, then updating together, avoiding visual flicker when different channels switch at different times.

> **Fade algorithm summary:** duty value changes in V-shape (1023→200→1023), staying/pausing at each extreme, forming "breathing" rhythm. Step 5 + delay 12ms makes fade smooth to human eye (~60fps equivalent update rate).

5.4 app_main() — Main Entry

Main function just three things: initialize hardware → create independent task to run breathing light → spin on its own. Why "independent task"? Because breathing light needs precise control of delay and duty updates. If directly written in main function, entire system gets blocked. After creating independent task, FreeRTOS scheduler automatically allocates CPU time, not interfering with each other.

```
void app_main(void)
{
    led_pwm_init();           // Initialize LEDC hardware

    // Create breath LED task
    // Parameters: func | name | stack 2048 | priority 1
    xTaskCreate(v_breath_led_task, "breath_led_task", 2048, NULL, 1, NULL);

    // Main task idles, doing nothing
    while (1)
        // Yield CPU every 1s
        vTaskDelay(pdMS_TO_TICKS(1000));
}
```

> **Is stack size 2048 bytes enough?** Breathing light task only uses a few local variables and LEDC API calls, 2048 bytes is more than sufficient. Can check actual stack usage at runtime via `uxTaskGetStackHighWaterMark()`.

---

6. Operation Flow

6.1 Build and Flash

> **For detailed flashing process, see: 1. Before Development → 3. Build and Flash**

6.2 Expected Phenomenon

- LED displays smooth **breathing light effect**
- Complete cycle about 6 seconds: OFF(2s) → fade in(2s) → fade out(2s) → OFF(2s) → repeat
- **No serial log output** (GPIO44 conflicts with UART0 TX, printing disabled)

6.3 Parameter Adjustment Guide

Parameter	Effect	Suggested Range
`BREATH_STEP`	Larger = faster fade	1~20 (too small = sluggish, too large = jumpy)
`BREATH_DELAY_MS`	Larger = slower fade	5~30ms
`OFF_HOLD_MS`	OFF state stay duration	500~5000ms
`VISIBLE_DUTY`	Brightest level (smaller = brighter)	0~300

---

7. Common Issues

Phenomenon	Cause	Solution
LED doesn't light up	duty=1023 (all high=OFF) / wrong pin	Check `VISIBLE_DUTY=200`, confirm waveform has low level
LED flickers/jitters	`BREATH_STEP` too large, duty jumps visibly	Reduce step (e.g., set to 2), appropriately increase delay
Serial monitor garbled	GPIO44 shares with UART0 TX, PWM interferes with serial	Normal! This experiment actively disables printing; use other GPIO for PWM
LED brightness uneven	Human brightness perception is non-linear (gamma curve)	Use gamma correction table instead of linear duty change
`ledc_fade_func_install` fails	Repeated installation / insufficient memory	Ensure called only once; check remaining DRAM
Breathing effect "stutters"	FreeRTOS scheduling delay / other tasks occupying CPU	Increase breath task priority or check for dead loop tasks